CS6886 — Course Project

A Unified Metal Kernel for MLX Bilinear Upsampling

Adhil Ahmed (da24s023) • November 1, 2026

GitHub: `https://github.com/starkAhmed43/cs6886-mlx-upsample`

1 The problem

What upsampling does

Upsampling increases the spatial size of an image-like tensor. For example, a 32×32 feature map becomes 64×64. An image-like tensor has height H, width W and C channels; we write its shape as $H \times W \times C$. Bilinear upsampling computes each new pixel as a weighted blend of its nearest input pixels, on each axis (Figure 1). Many vision models use it: segmentation decoders, depth decoders, mask heads and video frame interpolation.

Linear and bilinear interpolation

Each output pixel maps back to a position in the input. On one axis, this position x_{map} usually falls between two input pixels x_0 and x_1. Let $f(x)$ be the input value at position x. Let the weight $t \in [0, 1]$ be the fraction of the distance from x_0 to x_1. Then

$$f(x_{map}) = (1 - t)\, f(x_0) + t\, f(x_1).$$

In two dimensions, the position P_{map} falls between four neighbouring input pixels. Let Q_{00}, Q_{10}, Q_{01}, Q_{11} be their values, and let t_x and t_y be the weights on the x and y axes. Each corner's weight is the product of its weights on the two axes:

$$f(P_{map}) = (1 - t_x)(1 - t_y)\, Q_{00} + t_x(1 - t_y)\, Q_{10} + (1 - t_x)\, t_y\, Q_{01} + t_x t_y\, Q_{11}.$$

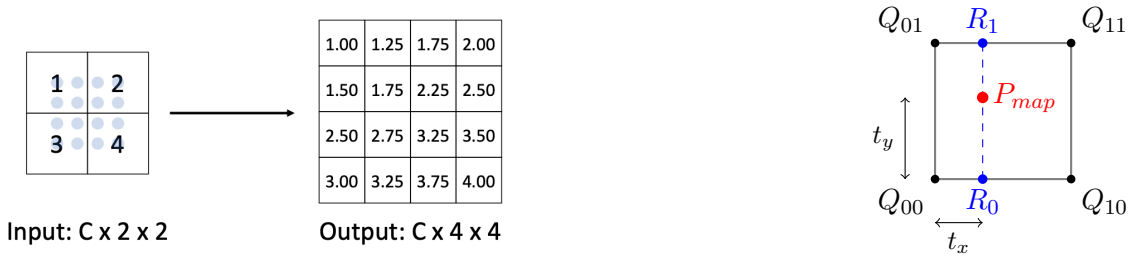


Figure 1: Left: bilinear upsampling of a small image (image from [1]). Right: the output point P_{map} and its four input neighbours. The blue points show the separable form.

Bilinear resize is separable

The 2D blend is two 1D blends. First, blend along x on the bottom and top edges to get R_0 and R_1, the blended values on the bottom and top edges. Then, blend the two results along y:

$$R_0 = (1 - t_x)Q_{00} + t_x Q_{10}, \qquad R_1 = (1 - t_x)Q_{01} + t_x Q_{11}, \qquad f(P_{map}) = (1 - t_y)R_0 + t_y R_1.$$

Substitution gives the 2D formula again. So each axis needs only an index table and a weight table. For each output index, the table gives the left neighbour, the right neighbour and t. There are two common ways to apply these tables on a GPU.

Method 1: gather

MLX's [2] `nn.Upsample` uses this method. For each axis, it gathers the left and the right neighbours with `take` and blends them. This does only the four useful multiply-adds for each output. But each step writes a full-size temporary array to memory, and the reads land at scattered addresses. The backward pass of a gather is a scatter-add into the input gradient, which is slower still.

Method 2: matmul

The tables also define a dense weight matrix for each axis. Let X be one $H \times W$ channel of the input, and Y the matching output channel. W_y has one row for each output row, and W_x has one row for each output column; row i of W_x holds $1 - t$ and t at the two neighbour columns of output column i, and zeros everywhere else. (Here W_x and W_y are matrices, not to be confused with the width W.) Then, for each channel, $Y = W_y \, X \, W_x^\top$. A matrix multiply reads memory in contiguous blocks and runs fast on a GPU. But most of each matrix is zeros. At large sizes, the wasted multiply-adds and the matrix memory grow quickly.

The bug

MLX issue #4435 [3] reports that a matmul resize beats `nn.Upsample` when it upscales. Table 1 shows our independent reproduction on an M1. The shapes come from RIFE [4], a video frame interpolation model. We write speed ratios as a multiple with an "x", e.g. 2x means twice as fast. Every ratio landed within 0.1x of the original report. The issue is still open.

Table 1: Issue #4435 reproduced on an M1 (float32, forward pass).

Shape	`nn.Upsample`	matmul resize	Ratio
$192 \times 320 \to 768 \times 1280$ ($\times 4$ up)	8.63 ms	2.26 ms	3.96x slower
$384 \times 640 \to 768 \times 1280$ ($\times 2$ up)	9.30 ms	4.72 ms	1.98x slower
$768 \times 1280 \to 192 \times 320$ ($\times 0.25$ down)	1.19 ms	2.05 ms	0.58x (faster)
$768 \times 1280 \to 384 \times 640$ ($\times 0.5$ down)	2.87 ms	4.14 ms	0.69x (faster)

So neither method wins everywhere. Gather wastes time on memory traffic and temporaries. Matmul wastes time on zeros. A direct kernel can do only the useful multiply-adds and still read memory in a GPU-friendly pattern.

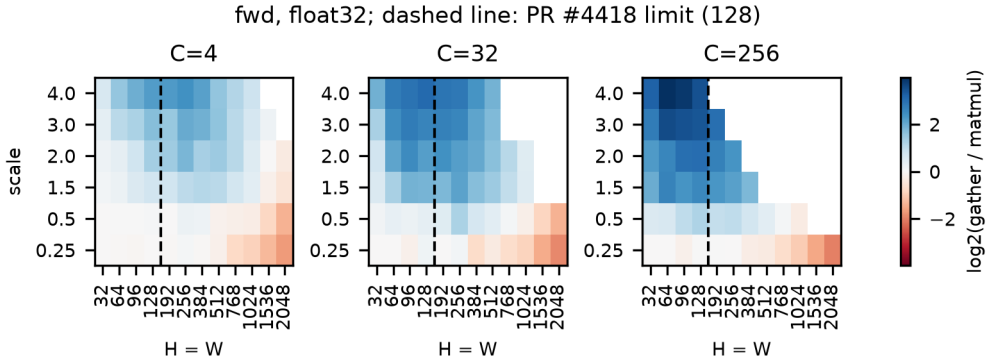
2 Prior work

PR #4418: This pull request [5] was the proposed fix for the issue. It adds the matmul method. It selects matmul only when the input is upscaled and at most 128 pixels. Otherwise, it selects gather. The proposal found three gaps. Nobody measured the 128-pixel limit, and the issue's own slow shapes are larger than 128 pixels. The PR has no gradient tests, and our tests found that gather and matmul gradients disagree in float16. The PR also had no tests on real model shapes. The PR was closed for inactivity on 2026-10-29.

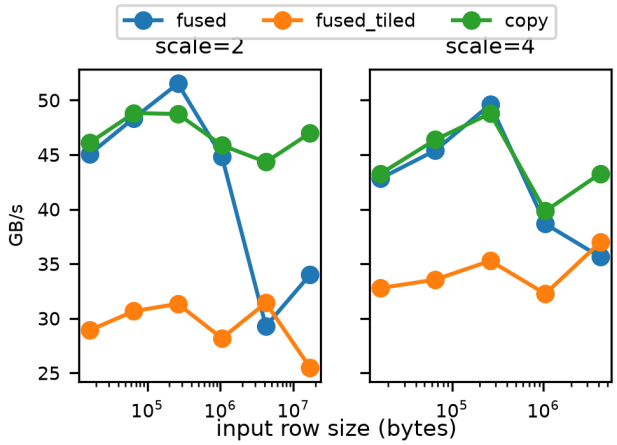
Other frameworks: PyTorch [6] has a dedicated CUDA kernel for `upsample_bilinear2d`. Its forward pass uses one thread for each output value. Its backward pass adds into the input gradient with atomic operations. MLX has no such kernel on Metal.

3 Idea and implementation

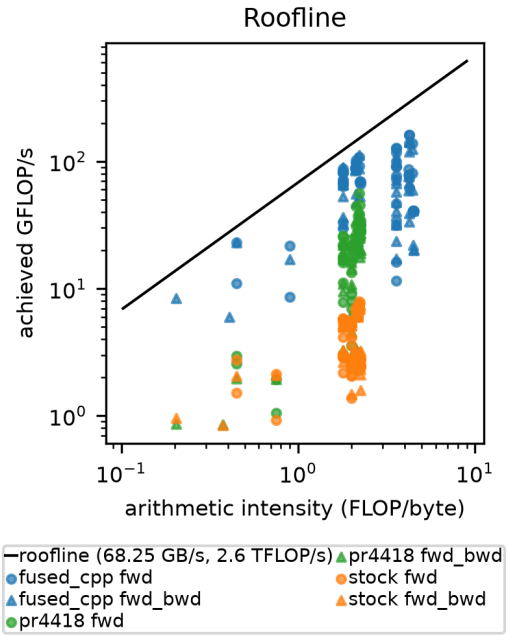
The idea is one Metal kernel for each pass. Each kernel reads only the inputs with a nonzero weight, writes each result once and makes no temporary arrays. We built it in five experiments.



(a) Experiment 01: gather time / matmul time, forward, float32. Blue means matmul is faster. The dashed line is the PR's 128-pixel limit.



(b) Experiment 03: bandwidth against input row size. The fused kernel tracks the copy limit up to 1 MiB rows.



(c) Experiment 05: roofline for all 318 rows (M1: 68.25 GB/s, 2.6 TFLOP/s). All variants are memory-bound.

Figure 2: Results of experiments 01, 03 and 05 on an M1. Source: the experiment folders in the project repository [7].

Each experiment is a folder in the repository with its scripts, raw results and a README. All timing comes from one Apple M1 with 16 GB of unified memory.

Experiment 01 – Where does matmul win? We timed gather, compiled gather, matmul and the PR's rule. We used a synthetic grid of 360 configurations and 28 shapes from seven real models. The models are SegFormer [8], UPerNet [9], DeepLabV3+ [10], PSPNet [11], DPT and Depth Anything [12, 13], SAM [14] and RIFE. For upscaling, matmul beat gather at almost every size. For 4 channels, the crossover was near 1536 pixels, far above 128. The PR's rule chose a method more than 10% slower than the best in 369 of 802 configurations (Figure 2a).

Experiment 02 – A fused forward kernel: The kernel runs one GPU thread for each output value. Each thread reads its four neighbours, blends them, and writes one value. Threads with consecutive IDs handle consecutive channels. MLX stores images as NHWC (batch N, height H, width W, channels C, with channels last), so neighbouring threads read neighbouring addresses. It matched stock MLX to 2.3×10^{-7} relative error on 402 workloads. It beat the best other method on 373 of them, with a median speedup of 2.5x. The ties had outputs under 0.7 ms. There, a fixed launch cost of 0.13–0.35 ms dominates. Caching the index tables for each shape removed a further 0.1 ms per call.

Experiment 03 – Does tiling help? The proposal planned tiling: load a block of input into threadgroup memory, so nearby threads share it. It did not help. The tiled kernel lost on all 49 real workloads, with a median of 0.70x the fused kernel's speed. A row-size test explains why. For input rows up to 1 MiB, the fused kernel already ran at 0.97–1.06x the speed of a plain copy kernel. A copy is the practical bandwidth limit, so no kernel can be much faster (Figure 2b). Tiling helps only above 1–4 MiB rows. The widest real row is 192 KiB, so we kept the simple kernel.

Experiment 04 – The backward pass: The backward pass maps the output gradient back to the input. A scatter would add each output into four inputs. This needs atomic additions, and float16 atomics lose precision. We use a gather instead. Because the resize is separable, the outputs that read one input row form a contiguous range. The host builds a small range table for each axis. For each input index k, the table gives the range $[lo(k), hi(k))$ of outputs that use it. The backward kernel runs one thread for each input value. Each thread loops over its two output ranges and adds the weighted gradients in float32. No two threads write the same address, so the kernel needs no atomics. It rounds to float16 only once, at the end. All gradient tests passed: parity with stock MLX, an adjoint test, finite differences and float16 accuracy.

Integration into MLX: We then moved both kernels into MLX as a C++ primitive on a fork branch [15]: `mx.fast.upsample_bilinear`. `nn.Upsample` calls it for bilinear mode without antialiasing. The index and weight rules are a C++ port of MLX's own rules, in float32. Function constants fix the sizes and channel count, at a cost of a 6 ms compile for each new shape. Its own vector-Jacobian product runs the backward kernel. It falls back to stock MLX on the CPU, for antialiasing, empty arrays and outputs above 2^{31} elements (the limit of a 32-bit index). A float16 input keeps a float16 output, unlike stock MLX's float32. The branch adds 222 lines of tests to MLX's `test_fast.py`, plus an export test. **We submitted the branch upstream as MLX pull request #4812 [16], which closes issue #4435.**

4 Evaluation

Method: Experiment 05 compares three variants through `nn.Upsample`: stock MLX 0.32.4, PR #4418 and our primitive. Each of the 28 real shapes runs at batch size 1 in float16 (inference) and at its training batch size in float32. We dropped 3 float32 rows with outputs above 256 MB, to avoid swap on a 16 GB machine. That leaves 53 workloads. Each workload, variant and pass ran in 10 fresh processes, in random order, and we report the median. A method wins only if it is faster by more than a 0.9x margin. Otherwise the result is a tie.

Speed: Table 2 gives the summary. Our primitive beats stock MLX on all 53 workloads in both passes. Against PR #4418, it wins 104 of 106 comparisons and ties 2. The largest gain is a Depth Anything block at $148 \times 148 \times 256$: 75.6 ms for stock MLX and 1.6 ms for ours, a 47x speedup. The smallest gain is 4.5x, on a small UPerNet block. Table 3 shows selected workloads. On all 16 rows with the issue's RIFE shapes, our primitive is 6.4–14.8x faster than stock MLX. A float16 output halves the bytes written, but this does not explain the gain. Float32 inputs, where all three

variants write float32, show 16.5x forward and 14.6x forward and backward over stock MLX.

Table 2: Speedup of our primitive over each baseline, across 53 real workloads.

	vs stock MLX			vs PR #4418		
Pass	geomean	median	min	geomean	median	min
Forward	16.6x	15.3x	4.5x	5.3x	3.9x	1.30x
Forward + backward	13.9x	13.1x	5.6x	4.7x	3.7x	0.99x

Table 3: Median time (ms) for selected workloads at batch size 1, float16. F is the forward pass; F+B is forward and backward.

Workload	Input, scale	Pass	Stock	PR #4418	Ours
RIFE up (issue)	$192 \times 320 \times 4$, $\times 4$	F	5.67	14.9	0.48
		F+B	11.6	25.2	1.82
RIFE down (issue)	$768 \times 1280 \times 4$, $\times 0.25$	F+B	5.26	5.19	0.74
SegFormer logits	$128 \times 128 \times 150$, $\times 4$	F+B	105	25.3	4.99
SAM masks	$256 \times 256 \times 3$, $\times 4$	F+B	9.53	20.4	1.69
PSPNet bin 1	$1 \times 1 \times 512$, $\times 60$	F+B	14.4	1.64	1.65

Roofline: Bilinear upsampling does about 9 FLOPs for each output value. Arithmetic intensity is the FLOPs a kernel performs for each byte it moves. For bilinear upsampling it is 0.2–4.5 FLOP/byte. The ridge point is the peak FLOP/s divided by the peak bandwidth; on the M1 this is about 38 FLOP/byte (2.6 TFLOP/s / 68.25 GB/s), so all three variants are limited by memory bandwidth (Figure 2c). We compute achieved bandwidth as the minimum bytes a correct kernel must move, divided by the time. Our primitive reaches a median of 45% of the 68 GB/s peak in the forward pass, and 32% with the backward pass. Its best row reaches 75%. Stock MLX reaches a median of 3%, and PR #4418 reaches 13%. So the speedup comes from less memory traffic, not from faster arithmetic.

Accuracy: In float32, the maximum relative gradient error is about 2×10^{-6} for all three variants. In float16, it is 6.2×10^{-4} for ours, 1.6×10^{-3} for PR #4418 and 2.7×10^{-2} for stock MLX. Stock MLX scatter-adds into a float16 array, so its rounding error grows with the number of outputs that read each input.

The two ties: Both ties are PSPNet backward passes with a 1×1 or 2×2 input and a scale of 60 or 30. The backward kernel starts one thread for each input value, so only 512 or 2048 threads run. Each thread loops over up to 3600 outputs, and most of the GPU is idle. Splitting each loop across several threads would likely fix this. We did not implement it.

Limitations: All numbers come from one M1. Other Apple GPUs can behave differently. The run-to-run spread has a median of 14%. It reaches 65% for some sub-millisecond rows. The PR #4418 baseline is the PR's own code. Above 128 pixels, its gather path is older than MLX 0.32.4's. So it is slower than stock MLX on some large shapes. No test reaches the 2^{31}-element fallback limit. We did not test the CUDA backend.

Conclusion: One direct kernel for each pass runs near the memory bandwidth limit and needs no atomics. It is 14–17x faster than stock MLX on average, gives more accurate float16 gradients, and wins or ties against both baselines.

References

[1] E. Wiener. AI notes: upsampling. Web page. URL: https://ericwiener.github.io/ai-notes/.

[2] A. Hannun, J. Digani, A. Katharopoulos, and R. Collobert. MLX: Efficient and flexible machine learning on Apple silicon. GitHub repository, 2023. URL: https://github.com/ml-explore/mlx.

[3] MLX issue #4435. GitHub issue. URL: https://github.com/ml-explore/mlx/issues/4435.

[4] Z. Huang et al. Real-time intermediate flow estimation for video frame interpolation. In *ECCV*, 2022.

[5] MLX pull request #4418. GitHub pull request. URL: https://github.com/ml-explore/mlx/pull/4418.

[6] A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.

[7] A. Ahmed. Project code and results. GitHub repository. Accessed 2026. URL: https://github.com/starkAhmed43/cs6886-mlx-upsample.

[8] E. Xie et al. SegFormer: Simple and efficient design for semantic segmentation with transformers. In *NeurIPS*, 2021.

[9] T. Xiao et al. Unified perceptual parsing for scene understanding. In *ECCV*, 2018.

[10] L.-C. Chen et al. Encoder-decoder with atrous separable convolution for semantic image segmentation. In *ECCV*, 2018.

[11] H. Zhao et al. Pyramid scene parsing network. In *CVPR*, 2017.

[12] R. Ranftl, A. Bochkovskiy, and V. Koltun. Vision transformers for dense prediction. In *ICCV*, 2021.

[13] L. Yang et al. Depth Anything: Unleashing the power of large-scale unlabeled data. In *CVPR*, 2024.

[14] A. Kirillov et al. Segment anything. In *ICCV*, 2023.

[15] A. Ahmed. MLX branch `upsample-fused`. GitHub branch. Accessed 2026. URL: https://github.com/starkAhmed43/mlx/tree/upsample-fused.

[16] MLX pull request #4812: fused Metal kernel for bilinear upsampling. GitHub pull request. URL: https://github.com/ml-explore/mlx/pull/4812.